Security Review

Cantina Competition

MORPHO MIDNIGHT

CANTINA

August 10, 2026

Contents

1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.
Learn more at cantina.security

1.2 Disclaimer

A competition provides a broad evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While competitions endeavor to identify and disclose all potential security issues, they cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities, therefore, any changes made to the code would require an additional security review. Please be advised that competitions are not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above matrix. High severity findings represent the most critical issues that must be addressed immediately, as they either have high impact and high likelihood of occurrence, or medium impact with high likelihood.

Medium severity findings represent issues that, while not immediately critical, still pose significant risks and should be addressed promptly. These typically involve scenarios with medium impact and medium likelihood, or high impact with low likelihood.

Low severity findings represent issues that, while not posing immediate threats, could potentially cause problems in specific scenarios. These typically involve medium impact with low likelihood, or low impact with medium likelihood.

Lastly, some findings might represent improvements that don't directly impact security but could enhance the codebase's quality, readability, or efficiency (Gas and Informational findings).

2 Security Review Summary

Morpho is a trustless and efficient lending primitive with permissionless market creation.

From May 29th to Jun 12th Cantina hosted a competition based on midnight. The participants identified a total of **11** issues in the following risk categories:

- High Risk: 0
- Medium Risk: 0
- Low Risk: 8
- Gas Optimizations: 0
- Informational: 3

2.1 Scope

The security review had the following components in scope for midnight on commit hash 7538c438:

```
src
├── Midnight.sol
├── interfaces
│   ├── ICallbacks.sol
│   ├── IERC20.sol
│   ├── IGate.sol
│   ├── IMidnight.sol
│   ├── IOracle.sol
│   └── IRatifier.sol
├── libraries
│   ├── ConstantsLib.sol
│   ├── EventsLib.sol
│   ├── IdLib.sol
│   ├── SafeTransferLib.sol
│   ├── TickLib.sol
│   └── UtilsLib.sol
├── periphery
│   ├── ConsumableUnitsLib.sol
│   ├── EcrecoverAuthorizer.sol
│   ├── MidnightBundles.sol
│   ├── TakeAmountsLib.sol
│   └── interfaces
│       ├── IEcrecoverAuthorizer.sol
│       ├── IERC20Permit.sol
│       ├── IMidnightBundles.sol
│       └── IPermit2.sol
└── ratifiers
    ├── EcrecoverRatifier.sol
    ├── SetterRatifier.sol
    ├── interfaces
    ├── IEcrecoverRatifier.sol
    │   └── ISetterRatifier.sol
    └── libraries
        └── HashLib.sol
```

3 Findings

3.1 Low Risk

3.1.1 A borrower can invalidate an exact liquidation transaction by front-running it with a tiny repayment

Severity: Low Risk.

Context: *(No context files were provided by the reviewer)*.

Submitted by rudeus33, also found by SiddiqX786, lazymio, ExtraCaterpillar, Daniel526, Cryptor, testnate, SOPROBRO, ABA, harry, SarveshLimaye and PotEater.

Summary: A borrower can front-run an exact debt-sized `liquidate` transaction with a one-wei repayment. The repayment lowers the stored debt immediately. The stale liquidation then attempts to subtract the old `repaidUnits` value from the smaller current debt and reverts.

An adaptive liquidator can recompute the current debt and retry successfully. The local proof of concept did not demonstrate bad debt escalation, reduced lender recovery, frozen funds, or attacker profit. This is therefore a Low findings, not a High/Medium finding.

Description: `Midnight.repay` allows a borrower or authorized operator to reduce debt by an arbitrary nonzero amount:

```
position[id][onBehalf].debt -= UtilsLib.toUint128(units);
marketState[id].withdrawable += UtilsLib.toUint128(units);
```

`Midnight.liquidate` later applies the liquidator-provided `repaidUnits` value against the current debt:

```
_marketState.withdrawable += UtilsLib.toUint128(repaidUnits);
_position.debt -= UtilsLib.toUint128(repaidUnits);
```

If a liquidator submits an exact full-debt liquidation and the borrower repays one wei before inclusion, the liquidation underflows and reverts. This can force liquidation bots to refresh quotes and resubmit transactions during volatile periods.

The proof of concept also establishes the limits of the observation:

- The one-wei repayment is backed by a real token transfer;
- It decreases rather than increases debt;
- An adaptive retry using the current debt fully liquidates the position;
- Lender withdrawable matches prompt control liquidation;
- At an identical lower oracle price, the tiny repayment does not increase socialized loss;
- An unrelated third party cannot dust-repay the borrower without authorization.

Impact Explanation: **Impact: Low**.

The demonstrated impact is wasted gas and delayed keeper execution for stale exact-input liquidation transactions. The borrower pays the repayment amount and transaction gas. A liquidator can recover by reading current debt and retrying.

No non-dust lender loss, bad-debt escalation, frozen recoverability, unfair liquidation, or attacker profit was demonstrated. Treat as Gas or Informational if the competition does not accept repairable transaction-invalidation grief as Low.

Why this is not High or Medium: The adaptive retry succeeds with equivalent lender recovery and zero `lossFactor` increase. The observation does not create a solvency break or non-dust economic loss.

Why this is still useful: The liquidation-specific trace is useful integration guidance: bots that submit exact debt-sized liquidations must refresh state and tolerate stale-input reverts.

Likelihood Explanation: **Likelihood: Medium**.

A borrower can observe a pending exact liquidation transaction and submit a one-wei repayment with a higher priority fee. The path uses ordinary ERC20 behavior and a normal borrower permission. Its practical usefulness is limited because liquidation bots can refresh state, batch retries, or use non-maximal liquidation amounts.

Dedup Note: Spearbit previously reported that `take()` calls can be forced to revert via front-running. This observation affects a different entrypoint and state transition: borrower `repay()` invalidates an exact `liquidate()` input. The common theme is mempool transaction invalidation, but the affected operation, attacker action, stale parameter, and mitigation are liquidation-specific.

Because the practical impact remains repairable gas grief, this packet is held for manual duplicate review and is not recommended for submission by default.

Proof of Concept: Github code: PoC_Hist_TinyRepayLiquidationGrief.t.sol.

Select **Local** in Cantina. The proof of concept uses the repository's Foundry harness and ordinary local ERC20 mocks. It does not require a live fork, RPC URL, or external deployment.

Preconditions:

- A borrower has `100e18` debt units.
- A liquidator prepares an exact full-debt liquidation using `repaidUnits = 100e18`.
- The borrower is authorized to repay its own debt.

Run command:

```
forge test --match-path
↪  test/asyam/poc/historical/PoC_Hist_TinyRepayLiquidationGrief.t.sol -vvvv
```

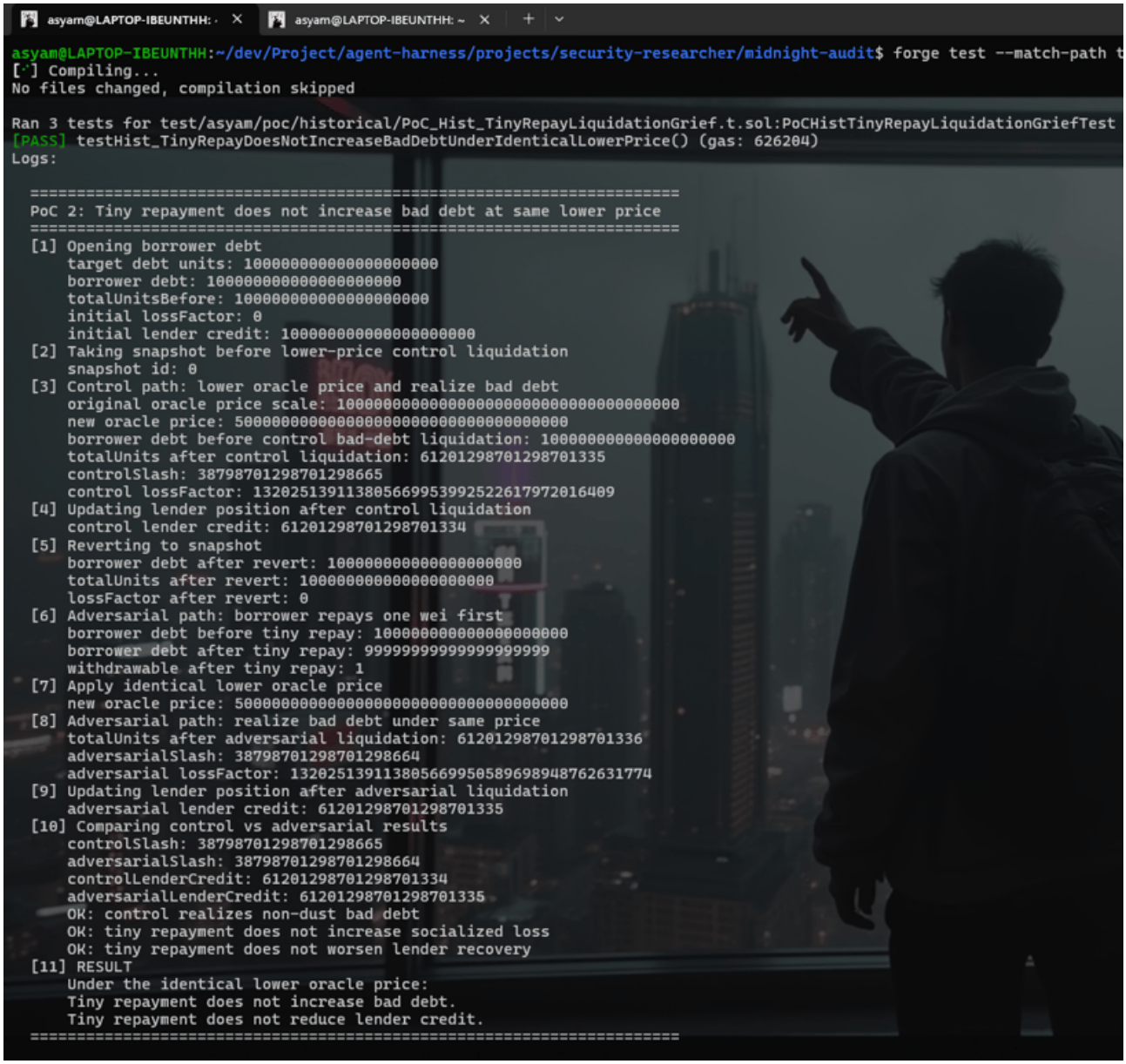
Verified locally on 2026-05-31:

```
Ran 1 test suite: 3 tests passed, 0 failed, 0 skipped
```

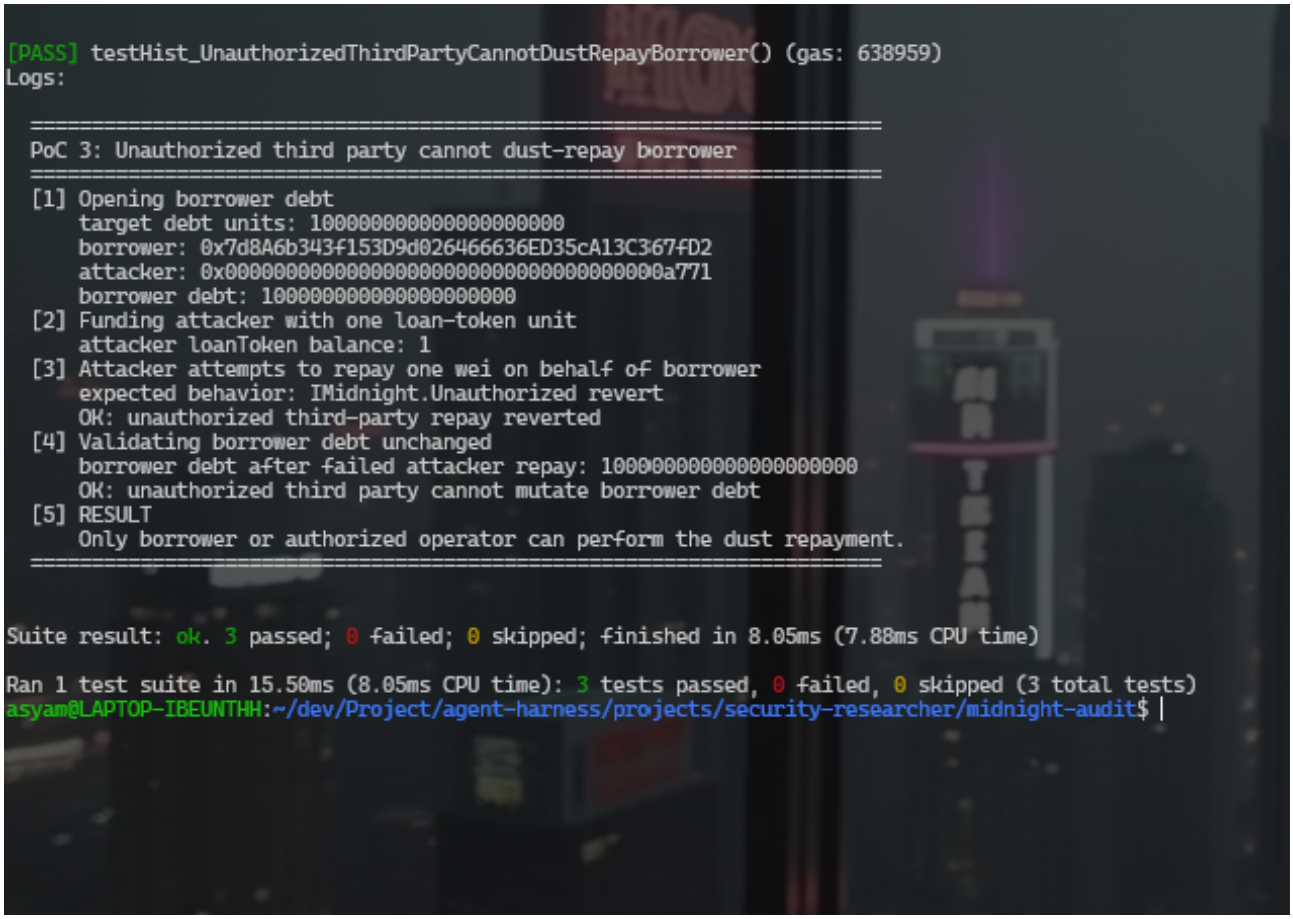
Core reproducer: The local test snapshots a control liquidation, then replays the same state with the borrower front-run:

```
function testHist_TinyRepayInvalidatesStaleExactLiquidationButAdaptiveRetryRecovers
↪  EquivalentValue() public {
    _openBorrowerDebt(UNITS);
    _fundLoanToken(LIQUIDATOR_ACTOR, UNITS);
    vm.warp(market.maturity + TIME_TO_MAX_LIF);

    uint256 snapshot = vm.snapshotState();

    vm.prank(LIQUIDATOR_ACTOR);
    midnight.liquidate(market, 0, 0, UNITS, borrower, true, LIQUIDATOR_ACTOR,
    ↪  address(0), hex"");

    uint256 controlWithdrawable = midnight.withdrawable(marketId);
    assertEq(midnight.debtOf(marketId, borrower), 0, "control liquidation repays all
    ↪  debt");
    assertEq(midnight.lossFactor(marketId), 0, "control liquidation realizes no bad
    ↪  debt");
```

```
    vm.revertToState(snapshot);

    vm.prank(borrower);
    midnight.repay(market, 1, borrower, address(0), hex"");

    vm.prank(LIQUIDATOR_ACTOR);
    vm.expectRevert(stdError.arithmeticError);
    midnight.liquidate(market, 0, 0, UNITS, borrower, true, LIQUIDATOR_ACTOR,
    ↪  address(0), hex"");

    uint256 currentDebt = midnight.debtOf(marketId, borrower);
    assertEq(currentDebt, UNITS - 1, "tiny repayment lowers debt by one wei");

    vm.prank(LIQUIDATOR_ACTOR);
    midnight.liquidate(market, 0, 0, currentDebt, borrower, true, LIQUIDATOR_ACTOR,
    ↪  address(0), hex"");

    assertEq(midnight.debtOf(marketId, borrower), 0, "adaptive liquidation repays
    ↪  current debt");
    assertEq(midnight.lossFactor(marketId), 0, "tiny repayment does not create bad
    ↪  debt");
    assertEq(midnight.withdrawable(marketId), controlWithdrawable, "adaptive retry
    ↪  restores equivalent recovery");

    uint256 lenderBefore = loanToken.balanceOf(lender);
    vm.prank(lender);
    midnight.withdraw(market, UNITS, lender, lender);
    assertEq(loanToken.balanceOf(lender), lenderBefore + UNITS, "lender can withdraw
    ↪  the full recovered claim");
}
```


asyam@LAPTOP-IBEUNTHH: ~
asyam@LAPTOP-IBEUNTHH:~/dev/Project/agent-harness/projects/security-researcher/midnight-audit$ forge test --match-path t
[⠊] Compiling...
No files changed, compilation skipped
Ran 3 tests for test/asyam/poc/historical/PoC_Hist_TinyRepayLiquidationGrief.t.sol:PoCHistTinyRepayLiquidationGriefTest
[PASS] testHist_TinyRepayDoesNotIncreaseBadDebtUnderIdenticalLowerPrice() (gas: 626204)
Logs:
PoC 2: Tiny repayment does not increase bad debt at same lower price
[1] Opening borrower debt
target debt units: 10000000000000000000
borrower debt: 10000000000000000000
totalUnitsBefore: 10000000000000000000
initial lossFactor: 0
initial lender credit: 10000000000000000000
[2] Taking snapshot before lower-price control liquidation
snapshot id: 0
[3] Control path: lower oracle price and realize bad debt
original oracle price scale: 100000000000000000000000000000000000
new oracle price: 50000000000000000000000000000000000
borrower debt before control bad-debt liquidation: 10000000000000000000
totalUnits after control liquidation: 61201298701298701335
controlSlash: 38798701298701298665
control lossFactor: 132025139113805669953992522617972016409
[4] Updating lender position after control liquidation
control lender credit: 6120129870129870134
[5] Reverting to snapshot
borrower debt after revert: 10000000000000000000
totalUnits after revert: 10000000000000000000
lossFactor after revert: 0
[6] Adversarial path: borrower repays one wei first
borrower debt before tiny repay: 10000000000000000000
borrower debt after tiny repay: 9999999999999999999
withdrawable after tiny repay: 1
[7] Apply identical lower oracle price
new oracle price: 50000000000000000000000000000000000
[8] Adversarial path: realize bad debt under same price
totalUnits after adversarial liquidation: 61201298701298701336
adversarialSlash: 38798701298701298664
adversarial lossFactor: 132025139113805669950589698948762631774
[9] Updating lender position after adversarial liquidation
adversarial lender credit: 61201298701298701335
[10] Comparing control vs adversarial results
controlSlash: 38798701298701298665
adversarialSlash: 38798701298701298664
controlLenderCredit: 61201298701298701334
adversarialLenderCredit: 61201298701298701335
OK: control realizes non-dust bad debt
OK: tiny repayment does not increase socialized loss
OK: tiny repayment does not worsen lender recovery
[11] RESULT
Under the identical lower oracle price:
Tiny repayment does not increase bad debt.
Tiny repayment does not reduce lender credit.

```
[PASS] testHist_TinyRepayInvalidatesStaleExactLiquidationButAdaptiveRetryRecoversEquivalentValue() (gas: 873354)
Logs:

  ========================================================================
  PoC 1: Tiny repayment invalidates stale exact liquidation
  ========================================================================
  [1] Opening borrower debt
      target debt units: 10000000000000000000
      borrower: 0x7d8A6b343f153D9d026466636ED35cA13C367fD2
      lender: 0xb012341CA6E91C00A290F658fbaA5211F2559fB1
      liquidator: 0x0000000000000000000000000000000000007002
      marketId:
  0x1f3f8c01fea9383e915942e33766d08c924221e3324931c353296d60035e6b6c
      borrower debt after opening: 10000000000000000000
      market totalUnits: 10000000000000000000
      market withdrawable: 0
      market lossFactor: 0
  [2] Funding liquidator with loan tokens
      liquidator loanToken balance: 10000000000000000000
  [3] Warping to maturity + TIME_TO_MAX_LIF
      current timestamp before warp: 1
      market maturity: 101
      TIME_TO_MAX_LIF: 900
      current timestamp after warp: 1001
  [4] Taking snapshot before control liquidation
      snapshot id: 0
  [5] Control path: liquidator repays exact full debt
      liquidate repaidUnits: 10000000000000000000
      borrower debt before control liquidation: 10000000000000000000
      borrower debt after control liquidation: 0
      control withdrawable: 10000000000000000000
      control lossFactor: 0
      control totalUnits: 10000000000000000000
      OK: control liquidation repays all borrower debt
      OK: control liquidation realizes no bad debt
  [6] Reverting to snapshot
      borrower debt after revert: 10000000000000000000
      withdrawable after revert: 0
      lossFactor after revert: 0
  [7] Borrower front-runs with one-wei repayment
      borrower debt before tiny repay: 10000000000000000000
      repay units: 1
      borrower debt after tiny repay: 99999999999999999999
      withdrawable after tiny repay: 1
  [8] Stale exact liquidation attempts to repay old full debt
      stale repaidUnits: 10000000000000000000
      current borrower debt: 99999999999999999999
      expected behavior: arithmetic underflow revert
      OK: stale exact liquidation reverted with arithmetic error
  [9] Validating current debt after tiny repayment
      currentDebt: 99999999999999999999
      expected currentDebt: 99999999999999999999
      OK: tiny repayment lowered debt by exactly one wei
  [10] Adaptive retry: liquidator repays recomputed current debt
      adaptive repaidUnits: 99999999999999999999
      borrower debt after adaptive liquidation: 0
      lossFactor after adaptive liquidation: 0
      withdrawable after adaptive liquidation: 10000000000000000000
      expected withdrawable: 10000000000000000000
      OK: adaptive liquidation repays current borrower debt
      OK: tiny repayment does not create bad debt
      OK: adaptive retry restores equivalent withdrawable recovery
  [11] Lender withdraws recovered claim
      lender loanToken balance before: 99999999990000000000000000000000
      lender loanToken balance after: 10000000000000000000000000000000
      lender balance delta: 10000000000000000000
      OK: lender can withdraw the full recovered claim
  [12] RESULT
      Tiny borrower repayment invalidates stale exact liquidation.
      Adaptive liquidation using current debt succeeds.
      No bad debt is created.
      Lender recovery remains equivalent to control.
  ========================================================================
```



.

What the Proof of Concept Proves:

Test	Proof
`testHist_TinyRepayInvalidatesStaleExactLiquidationButAdaptiveRetryRecoversEquivalentValue`	A borrower repayment of `1 wei` causes the stale exact liquidation to revert with arithmetic underflow. A liquidation recomputed against current debt succeeds and restores the same lender recovery as the prompt control.
`testHist_TinyRepayDoesNotIncreaseBadDebtUnderIdenticalLowerPrice`	At an identical lower oracle price, the one-wei repayment does not increase socialized loss or reduce lender credit relative to control.
`testHist_UnauthorizedThirdPartyCannotDustRepayBorrower`	An unrelated third party cannot mutate borrower debt because `repay` reverts with `Unauthorized`.

Trace interpretation: The first post-repayment liquidation is stale: it still submits `UNITS`, while the borrower's current debt is `UNITS - 1`. The subtraction in `Midnight.liquidate` reverts. The subsequent liquidation reads current debt, repays `UNITS - 1`, leaves debt at zero, leaves `lossFactor` at zero, and allows the lender to withdraw the full recovered claim.

Recommendation: Document that liquidation integrations should refresh debt immediately before submission and tolerate stale transaction reverts. If the protocol wants to reduce this operational surface, consider a bounded liquidator slippage mechanism or a repay-all sentinel that resolves against current debt while preserving the liquidator's minimum seized collateral constraint.

One possible integration-facing API is an explicit repay-all sentinel:

```
uint256 effectiveRepaidUnits = repaidUnits == type(uint256).max
    ? _position.debt
    : repaidUnits;
```

Any sentinel implementation must preserve the liquidator's seized-collateral slippage bound and existing RCF checks.

Morpho: Acknowledged.

Cantina Managed: Acknowledged.

3.1.2 Continuous-fee increases can overcharge resting buy-offer makers

Severity: Low Risk.

Context: *(No context files were provided by the reviewer).*

Submitted by yuzeguitarist, also found by fuddleyu.

Severity: Medium.

Likelihood: Medium.

The issue is triggerable when a market's continuous fee is increased while an already-ratified buy offer remains fillable. Resting buy offers without a defensive callback are realistic because the core offer format does not include a maximum acceptable pending-fee increase.

Impact: High.

A buy-offer maker can receive newly minted credit carrying a much larger pending continuous fee than was in force when the offer was authorized. In a long-maturity market, the unexpected pending fee can approach the full credit amount, so most of the eventual repayment is claimable as continuous fee instead of being withdrawable by the lender.

Description:

Summary: Resting buy offers do not bind the market's mutable continuous fee, but `take` computes the buyer's new `pendingFee` from the current fee at fill time. If the fee is increased after the maker authorizes a buy offer and before a borrower fills it, the passive maker receives credit encumbered by the new higher fee even though the signed offer did not authorize that fee burden.

Root Cause

- Path: `src/Midnight.sol`.
- Contract/library: `Midnight`.
- Function: `take`.
- Failing assumption: a ratified buy offer remains valid even if the mutable market continuous fee changes before the offer is consumed.
- Why the check/accounting/order is wrong: `take` computes `buyerPendingFeeIncrease` from `_marketState.continuousFee` at execution time and does not compare it with any maker-authorized maximum. For resting buy offers, the harmed buyer is the maker, not the transaction caller, so the caller can consume the offer after a fee increase and impose a larger pending fee on the maker's new credit.

Vulnerability Details

1. Preconditions.
    - A market exists with a low continuous fee.
    - A lender authorizes a buy offer in that market.

- The offer has no callback that rejects unexpected `pendingFeeIncrease`.
- The market continuous fee is increased before the offer expires.

2. Entry point.
   - A borrower calls `take` on the unchanged buy offer.
3. State before.
   - The offer terms still match the ratified offer data.
   - The market's `continuousFee` is now higher than it was when the maker authorized the offer.
4. Attacker action.
   - The borrower fills the resting buy offer after the fee increase.
5. Faulty internal transition.
   - `take` computes `buyerPendingFeeIncrease` using the current higher `_marketState.continuousFee`.
   - The maker's credit is increased and the unexpectedly large pending fee is attached to that credit.
6. State after.
   - The maker owns credit, but most of that credit can accrue to `continuousFeeCredit` by maturity.
7. Why the result violates the intended protocol behavior.
   - The offer binds market identity, price, caps, receiver, callback, and ratifier, but it does not bind the fee burden imposed on newly minted credit. The maker can therefore receive materially worse credit terms than the authorized offer expressed.

Impact Details

- The lender's credit can be encumbered by an unexpectedly large pending fee.
- At maturity, a large share of the borrower's repayment can be claimed as continuous fee instead of withdrawn by the lender.
- The external filler does not need to change the offer, bypass ratification, or use a callback; the stale offer is consumed through the normal `take` path.
- The impact is bounded by the protocol's maximum continuous fee and remaining time to maturity, but for maximum-duration markets it can approach the full credit amount.

Mutation Testing Notes

- Filling the same offer without increasing `continuousFee` leaves the maker's `pendingFee` at zero.
- A defensive buy callback that rejects `pendingFeeIncrease > 0` makes the stale fill revert, showing that the missing maker-authorized bound is the relevant control.
- Reducing the maturity or using a smaller fee reduces the loss, while maximum maturity and maximum continuous fee make the pending fee exceed 99% of the new credit.

Recommended Mitigation

Bind the continuous-fee burden to the authorized offer. For example, add a maker-authorized `maxPendingFeeIncrease` or fee-rate bound to the offer data and enforce `buyerPendingFeeIncrease <= maxPendingFeeIncrease` in `take`. Alternatively, include a fee epoch in ratified offer data and increment that epoch on continuous-fee changes so old buy offers must be refreshed before they can mint new credit.

Morpho: Acknowledged.

Cantina Managed: Acknowledged.

3.1.3 Signed offers cannot bind an intended taker

Severity: Low Risk.

Context: *(No context files were provided by the reviewer).*

Submitted by yuzeguitarist.

Severity: Medium.

Likelihood: Medium.

The issue is realistically triggerable when a maker shares a preferential signed quote with one counterparty, or when the intended counterparty's fill transaction exposes the signed offer and ratifier data before inclusion. The official `EcrecoverRatifier` validates the maker signature, but neither the signed offer nor the ratifier call binds the taker that is allowed to consume the quote.

Impact: Medium.

An unintended taker can consume a signed offer before the intended taker, forcing the maker's liquidity or credit sale to settle with the wrong counterparty. The maker receives the signed economic terms, but loses counterparty control for private or preferential quotes, and the intended taker is blocked once the shared offer cap is consumed.

Description:

Summary: Signed offers are bearer instruments: `take` receives `taker` as a separate call argument, but `HashLib.hashOffer` does not include an intended taker and `IRatifier.isRatified` is only passed the `Offer` plus `ratifierData`. As a result, a valid `EcrecoverRatifier` signature for a quote intended for one borrower or lender can be copied and filled by any other eligible taker before the intended counterparty, consuming the maker's offer under an unintended counterparty relationship.

Root Cause

- Path: `src/Midnight.sol`.
- Contract/library: `Midnight`.
- Function: `take`.
- Failing assumption: Ratification of an `Offer` is enough to authorize the actual taker that consumes it.
- Why the check/accounting/order is wrong: `take` checks `IRatifier(offer.ratifier).isRatified(offer, ratifierData)` before applying the fill, but the ratifier does not receive the `taker` argument. The official EIP-712 offer hash in `src/ratifiers/libraries/HashLib.sol` also omits an intended taker field, so `src/ratifiers/EcrecoverRatifier.sol` can only prove that the maker signed the quote, not that the caller is the intended counterparty.

Vulnerability Details

1. Preconditions.
    - A maker creates a preferential signed offer using `EcrecoverRatifier`.
    - The maker intends the quote for one specific counterparty.
    - The signed offer and `ratifierData` are visible to another eligible taker before the intended taker consumes it.
2. Entry point.
    - The unintended taker calls `take` with the same signed `Offer` and `ratifierData`, but passes their own address as `taker`.
3. State before.

- The maker has authorized `EcrecoverRatifier`.
- The offer's group has enough unconsumed capacity.
- The unintended taker can satisfy the normal protocol requirements, such as collateral for a borrow-side fill or loan tokens for a lend-side fill.

4. Attacker action.
   - The unintended taker copies the signed ratifier data and fills the offer first.
5. Faulty internal transition.
   - `take` verifies the maker signature over `Offer`, but the verified data does not include the actual `taker`.
   - `take` then consumes the maker's group and settles credit/debt with the unintended taker.
6. State after.
   - The unintended taker receives the loan assets or credit exposure.
   - The intended counterparty can no longer fill if the cap was fully consumed.
7. Why the result violates the intended protocol behavior.
   - A ratified private quote should not be consumable by a different counterparty solely because the signed payload became visible.

Impact Details

- Preferential maker quotes cannot be safely restricted to the intended borrower or lender.
- A front-running taker can consume scarce offer capacity and block the intended taker.
- The maker can be forced into exposure to a different borrower or lender than intended.
- The issue affects signed offers using the official `EcrecoverRatifier`; custom ratifiers also cannot enforce taker binding through `IRatifier` because the interface does not pass `taker`.

Mutation Testing Notes

- The intended borrower can fill successfully when they are first, confirming that the signature and market setup are otherwise valid.
- The same missing taker binding affects sell offers: an unintended lender can fill the borrower's signed sell offer before the intended lender.
- The issue does not depend on stale authorization, revoked signers, zero prices, callbacks, or bad debt; the tested signatures are current and valid.

Recommended Mitigation

Add optional taker binding to the signed and ratified offer path. Practical options include:

- Add an `address taker` or `address allowedTaker` field to `Offer`, with `address(0)` meaning unrestricted.
- Include that field in `HashLib.hashOffer`.
- In `take`, require the field to be unset or equal to the actual `taker`.
- Alternatively, extend `IRatifier.isRatified` to receive the actual `taker`, `msg.sender`, and fill amount so ratifiers can enforce counterparty and fill policies.
- Add regression tests where a signed offer intended for one taker cannot be filled by another taker using the same ratifier data.

Morpho: Acknowledged.

Cantina Managed: Acknowledged.

3.1.4 `MidnightBundles` `minUnits` check can pass before callback-created loss slashes received credit

Severity: Low Risk.

Context: *(No context files were provided by the reviewer).*

Submitted by Tradi3, also found by hartmade, Bullshido, kalva and ACCURATEBENNY.

Proof of concept gist: 0545d4660a73f661cca91292ad56b5ed.

Summary: `MidnightBundles.buyWithAssetsTargetAndWithdrawCollateral()` says the taker will gain at least `minUnits`, but the check counts the raw units returned by `Midnight.take()` before the taker's position is synced against the market's current `lossFactor`.

A sell-offer maker can use its normal seller callback to liquidate a different bad-debt borrower in the same market. That liquidation increases `lossFactor` after the buyer has been credited, but before the bundle returns. The bundle still counts `unitsToTake` toward `filledUnits` and passes `require(filledUnits >= minUnits)`.

The result is a successful bundle call where the taker pays for 100 units and passes `minUnits = 100e18`, but after `updatePosition()` the taker's credit is only about 30.6 units.

Affected code: Repository: `morpho-org/midnight` Commit: `7538c438513622721e23a94676b93a335b83dace` Files in scope: `src`.

- `src/periphery/MidnightBundles.sol:178` documents: "The taker will gain at least minUnits.".
- `src/periphery/MidnightBundles.sol:208-225` computes `unitsToTake`, adds that value to `filledUnits`, and checks `filledUnits >= minUnits`.
- `src/Midnight.sol:408-418` credits the buyer and updates market units before callbacks.
- `src/Midnight.sol:458-476` runs the seller callback before `take()` returns.
- `src/Midnight.sol:626-640` raises `lossFactor` when a liquidation realizes bad debt.
- `src/Midnight.sol:797-819` applies that `lossFactor` when the buyer position is synced.

Details: The bundle checks the amount it asked `take()` to fill, not the amount of credit the taker can actually keep after all callback effects in the same transaction.

In a sell-offer take:

1. `Midnight.take()` credits the buyer.
2. `Midnight.take()` pays the seller.
3. `Midnight.take()` calls the seller callback.
4. The seller callback liquidates a different same-market borrower with bad debt.
5. `liquidate()` increases the market `lossFactor`.
6. `take()` returns to `MidnightBundles`.
7. `MidnightBundles` adds the pre-slash `unitsToTake` to `filledUnits`.
8. `MidnightBundles` passes `filledUnits >= minUnits`.
9. When the taker's position is updated, the newly bought credit is slashed by the callback-created loss.

The seller itself is not being liquidated, so the seller liquidation lock does not stop this path. The callback targets a different borrower in the same market.

Impact: A taker can rely on the official periphery slippage floor and still receive less credit than `minUnits`.

In the proof of concept:

- Taker pays 100 loan tokens.
- Taker sets `minUnits = 100e18`.
- Bundle returns successfully.
- Raw taker credit after the bundle is `100e18`.
- After syncing the taker's position, credit is only `30.600649350649350667e18`.

This is direct value loss for users taking offers through the in-scope periphery. I would rate it Medium: it needs a sell offer with a seller callback and a same-market borrower that can be liquidated into bad debt, but those are protocol-supported states and no privileged role is required.

Likelihood: Medium.

Required conditions:

- The taker uses `MidnightBundles.buyWithAssetsTargetAndWithdrawCollateral()`;
- The offer is a sell offer with a seller callback;
- There is a different same-market borrower whose liquidation realizes bad debt;
- The seller callback calls `liquidate()` before returning.

The path does not require token reentrancy, a malicious ERC20, governance, or privileged access.

Proof of Concept: Copy `BundlesMinUnitsSlashedBySellerCallbackPoC.t.sol` from the gist into `test/poc/` and run:

```
forge test --match-contract BundlesMinUnitsSlashedBySellerCallbackPoC -vv
```

Expected output:

```
Ran 1 test for test/poc/BundlesMinUnitsSlashedBySellerCallbackPoC.t.sol:BundlesMinU⌋
↪  nitsSlashedBySellerCallbackPoC
[PASS] test_bundleMinUnitsPassesBeforeFreshCreditIsSlashedBySellerCallback() (gas:
↪  1266731)
Logs:
  lender paid loan tokens: 100000000000000000000
  bundle minUnits argument: 100000000000000000000
  raw lender credit after bundle: 100000000000000000000
  post-update lender credit: 30600649350649350667
  market lossFactor: 23615375301737206670698215319026542161 5
```

The last assertion is:

```
assertLt(postUpdateCredit, units, "minUnits guarantee was checked before
↪  callback-created loss");
```

Known issue / duplicate check: I checked the whitepaper, README, in-code comments, public audit text available in the repo context, local git history, GitHub code search, and public web search.

Closest public items I found:

- Blackthorn L-10 covers the accepted trade-off around optimistic bad-debt socialization. It does not cover `MidnightBundles` passing `minUnits` before a same-transaction seller callback raises `lossFactor`.

- Blackthorn L-24 mentions `MidnightBundles.buyWithAssetsTargetAndWithdrawCollateral()`, but that issue is about tick 0 pricing and inverse fee math. This proof of concept uses a normal high tick and callback-created bad debt.
- TrustSec L-5 covers continuous-fee slippage in bundle buy functions. This proof of concept does not rely on a `continuousFee` change; the bundle satisfies `minUnits` on raw units before same-transaction bad-debt realization changes the taker's effective credit.
- TrustSec L-6 covers sell-bundle exposure bounds and notes that bad debt can be realized during bundle callbacks. This proof of concept is the buy-side minimum-receive variant: `buyWithAssetsTargetAndWithdrawCollateral()` returns successfully even though the taker's post-update credit is far below the `minUnits` floor.
- Spearbit 5.4.6 covers loss-index inflation edge cases. This proof of concept does not depend on gradual inflation or a pre-existing inflated loss state; the loss is created after the bundle has counted the fill.
- The seller liquidation lock only protects the seller from being liquidated during `take()` callbacks. This proof of concept liquidates a different borrower in the same market.

Search strings included:

- `buyWithAssetsTargetAndWithdrawCollateral minUnits lossFactor`
- `MidnightBundles UnitsTooLow`
- `seller callback lossFactor Morpho Midnight`
- `MidnightBundles lossFactor`
- `callback liquidate other borrower same market`

I did not find a public report matching this periphery `minUnits` bypass.

Recommendation: Make the bundle's `minUnits` check use post-callback, post-loss-factor credit.

Possible fixes:

- Sync the taker's position after each take, then count the actual credit delta instead of `unitsToTake`;
- Record the market `lossFactor` before a bundle take and revert if it changes before the bundle performs its `minUnits` check;
- Block same-market bad-debt liquidations during `take()` callbacks, not only liquidation of the seller.

If the intended guarantee is only "raw units were filled before later loss-factor updates", the NatSpec and UI/integrator guidance should say that clearly, because the current wording reads like an economic minimum receive check.

Morpho: Acknowledged.

Cantina Managed: Acknowledged.

3.1.5 Malicious takers will make lender recovery uneconomic for lenders

Severity: Low Risk.

Context: *(No context files were provided by the reviewer).*

Submitted by Agontuk1, also found by zpan, MaanVad3r, Peter3144 and legendweb3.

Summary: Missing minimum fill and minimum borrower debt checks in `take()` will cause an economic recovery denial of service for lenders as a malicious taker will split one large lender buy offer into many sybil borrower positions whose individual liquidation reward is lower than the cost to liquidate them.

Root cause: In src/Midnight.sol:337-414, `take()` lets the taker choose any `units` amount and then writes `sellerDebtIncrease` to the borrower without checking a minimum fill size or minimum resulting borrower debt.

In src/interfaces/IMidnight.sol:21-36, `Offer` only has maximum caps, `maxUnits` and `maxAssets`. It has no maker-signed minimum fill, no `minUnits`, and no `minBorrowUnits`.

In src/Midnight.sol:655-667, `rcfThreshold` is only used inside `liquidate()` as a residual dust check. It does not stop a new borrower position from being created below that same threshold.

Description: `Midnight` lets users lend and borrow by trading credit units through `take()`. When an `Offer` has `buy == true`, the maker is the lender. The taker becomes the borrower if the taker has no credit to sell. The lender signs an offer with a total cap, then any taker can choose the `units` amount for each fill.

The market also has a dust parameter, `rcfThreshold`. This is meant to handle liquidation sizes that are too small compared to gas. The whitepaper says the dust threshold should be set so that positions above it can be "liquidated profitably given typical gas costs on the host chain." The code comment in `Midnight.sol` says the same idea: the recovery close factor is deactivated when the remaining liquidation would be too small compared to gas.

> **i Note:**
> This report is not about zero-price debt, zero-input liquidation, callback ordering, bad-debt socialization, or a reverting oracle. The proof of concept uses a normal full-price buy offer, real collateral, no callbacks, and no oracle failure.

The problem is that the dust logic is only used after the bad position already exists. `take()` does not check whether the taker's new debt is above `rcfThreshold`, or above any maker-defined minimum. So one large lender offer can be filled in many tiny pieces by many attacker-controlled borrower addresses.

```
363: uint256 buyerAssets = offer.buy ? units.mulDivDown(buyerPrice, WAD) :
↪  units.mulDivUp(buyerPrice, WAD);
364: uint256 sellerAssets = offer.buy ? units.mulDivDown(sellerPrice, WAD) :
↪  units.mulDivUp(sellerPrice, WAD);
//...
371: newConsumed = consumed[offer.maker][offer.group] += units; // @audit only total
↪  consumption is capped
372: require(newConsumed <= offer.maxUnits, ConsumedUnits());
//...
382: uint256 buyerCreditIncrease = UtilsLib.zeroFloorSub(units, buyerPos.debt);
383: uint256 sellerCreditDecrease = UtilsLib.min(units, sellerPos.credit);
384: uint256 sellerDebtIncrease = units - sellerCreditDecrease; // @audit no minimum
↪  debt or minimum fill check
//...
410: buyerPos.credit += UtilsLib.toUint128(buyerCreditIncrease);
//...
414: sellerPos.debt += UtilsLib.toUint128(sellerDebtIncrease); // @audit dust debt
↪  is stored as a normal borrower position
```

The lender receives one normal `credit` balance. But the matching debt is now split across many borrowers. After maturity, `liquidate()` is the path that turns defaulted debt into `withdrawable` loan tokens for lenders. That function takes only one `borrower` argument, so each dust borrower needs a separate transaction.

```
581: function liquidate(
582:     Market calldata market,
583:     uint256 collateralIndex,
584:     uint256 seizedAssets,
```

```
585:     uint256 repaidUnits,
586:     address borrower, // @audit one borrower per liquidation call
587:     bool postMaturityMode,
588:     address receiver,
589:     address callback,
590:     bytes calldata data
591: ) external returns (uint256, uint256) {
```

The intended dust threshold does not protect the lender here. It only changes the close-factor rule inside `liquidate()`.

```
655: if (!postMaturityMode) {
656:     uint256 lltv = market.collateralParams[collateralIndex].lltv;
//...
662:     require(
663:         repaidUnits <= maxRepaid
664:             ||
↪  _position.collateral[collateralIndex].mulDivDown(liquidatedCollatPrice,
↪  ORACLE_PRICE_SCALE)
665:                 .mulDivDown(WAD, lif).zeroFloorSub(maxRepaid) <
↪  market.rcfThreshold, // @audit liquidation-time dust check only
666:         RecoveryCloseFactorConditionsViolated()
667:     );
668: }
```

The highest impact case is a large lender buy offer in a market with a sane nonzero `rcfThreshold`. The attacker fills it through many sybil borrowers, each below `rcfThreshold`. When maturity passes, every borrower is technically liquidatable. But each liquidation reward is too small to pay for its own gas. Rational liquidators skip the accounts. The lender must either pay for many bad transactions, or wait with liquidity stuck. If collateral price falls during that delay, the issue can move from recovery delay to lender loss.

There is also a lesser impact that applies without a price fall. The lender cannot withdraw until the borrower debts are repaid or liquidated. So the lender's position is forced into a bad operational state where recovery depends on subsidized liquidation instead of normal incentives.

Attack Path:

1. Lender signs a normal buy `Offer` with `offer.buy == true`, `offer.maxUnits` set to a large amount, and `offer.tick == MAX_TICK`.
2. Attacker prepares many sybil borrower addresses and supplies enough collateral for each one by calling `supplyCollateral()`.
3. Each sybil borrower calls `take()` with a very small `units` value, below the market `rcfThreshold`.
4. `take()` increases the lender's `credit`, increases `consumed[lender][group]`, and writes the small `sellerDebtIncrease` to that sybil borrower.
5. The attacker repeats step 3 until the lender offer is consumed across many borrower addresses.
6. At maturity, the lender has one aggregate `credit` position, but `withdrawable` is still zero because the borrower debts are unpaid.
7. A liquidator must call `liquidate()` once per sybil borrower to recover the lender funds.
8. Each `liquidate()` call has a positive reward, but the reward is smaller than gas. Rational liquidators do not clear the dust accounts.
9. The lender is left with uneconomic recovery. If collateral falls before someone subsidizes liquidation, the lender can take real loss.

Impact: A malicious taker can turn one lender offer into many below-threshold borrower debts that are not profitable to liquidate one by one. This blocks normal recovery and traps lender liquidity after maturity. With later collateral price movement, the blocked recovery can become actual lender loss.

Mitigation: Add a minimum debt or minimum fill invariant to `take()`. The safest fix is to add a maker-signed `minUnits` or `minDebtAfterTake` field to `Offer`, and also enforce a market-level minimum when `sellerDebtIncrease > 0`, such as requiring the new borrower debt to be at least `rcfThreshold`.

```
  diff --git a/src/interfaces/IMidnight.sol b/src/interfaces/IMidnight.sol
  @@
+       error DebtBelowDustThreshold();

  diff --git a/src/Midnight.sol b/src/Midnight.sol
  @@
            uint256 sellerDebtIncrease = units - sellerCreditDecrease;
+           if (sellerDebtIncrease > 0) {
+               require(sellerPos.debt + sellerDebtIncrease >=
 ↪  offer.market.rcfThreshold, DebtBelowDustThreshold());
+           }
```

Morpho: Morpho acknowledged the finding as valid and noted it was not well documented. The `rcfThreshold` parameter only prevents the recovery close factor from creating progressively smaller positions during liquidation; it does not prevent dust debt from being created at `take()` time, leaving Midnight no more protected against dust-position fragmentation than Morpho Blue.

Cantina Managed: Acknowledged.

3.1.6 Liquidating Highest-LLTV Collateral Degrades Health Ratio and Amplifies Bad Debt Risk

Severity: Low Risk.

Context: *(No context files were provided by the reviewer).*

Submitted by D4N0, also found by Elizabeth, SarveshLimaye, strongbytes1337, EddiePumpin, peterVN, Riceee and Nabil.

Summary: In Midnight's multi-collateral liquidation, the RCF formula uses only the **individual** liquidated collateral's LLTV to compute `maxRepaid`, while `maxDebt` reflects the **full basket**. When a position is sufficiently underwater (`maxDebt/debt < lif × lltv_high`), a partial liquidation of the highest-LLTV collateral **decreases** the health ratio rather than improving it. Repeated partial liquidations of the premium collateral progressively degrade the basket's average LLTV, leaving lower-quality collateral backing a disproportionately large remaining debt and increasing the probability of bad debt by up to 58% relative compared to liquidating in the correct (lowest-LLTV-first) order.

Description:

Root Cause: `Midnight.sol:655-661` computes `maxRepaid` for the RCF using the **single liquidated collateral's** `lltv`:

```
uint256 lltv = market.collateralParams[collateralIndex].lltv;   // ← per-collateral
uint256 maxRepaid = lltv < WAD
    ? (_position.debt - maxDebt).mulDivUp(WAD * WAD, WAD * WAD - lif * lltv)
    : type(uint256).max;
```

Meanwhile, `maxDebt` (lines 607–619) is the sum across **all activated collaterals**:

```
while (_collateralBitmap != 0) {
    // ...
    maxDebt += _collateral.mulDivDown(price,
     ↪  ORACLE_PRICE_SCALE).mulDivDown(_collateralParam.lltv, WAD);
```

There is no protocol-level enforcement requiring any ordering of which collateral index to liquidate. The protocol explicitly acknowledges this in its NatSpec :

> *"The oracle-quoted liquidator incentive might not be constant across activated collaterals. Hence, liquidators may have a preference order over collaterals when liquidating."*.

The Mathematical Condition for Health Degradation: A partial liquidation of collateral `j` with repayment `r` changes the health ratio as follows:

```
new health ratio = (maxDebt - r × lif_j × lltv_j) / (debt - r)

Health DEGRADES (new ratio < old ratio) when:
  lif_j × lltv_j  >  maxDebt_basket / debt
```

For Midnight's allowed LLTV tiers with `LIQUIDATION_CURSOR_LOW = 0.25` :

Collateral lltv	lif (cursor=0.25)	lif × lltv	Degrades health when
0.860 (LLTV_3)	≈1.036	≈0.891	debt > maxDebt / 0.891
0.770 (LLTV_2)	≈1.061	≈0.817	debt > maxDebt / 0.817

In a representative 2-collateral market with 1000 tokens each at `lltv=[0.77, 0.86]` :

- `maxDebt = 770 + 860 = 1630` at oracle price = 1.
- `debt = 1900` → health ratio = 0.858.
- Degradation condition for high-lltv: `0.891 > 0.858` ✓.

Partial liquidation of the high-lltv collateral decreases the health ratio from 0.858 to 0.856.
Partial liquidation of the low-lltv collateral increases the health ratio from 0.858 to 0.860.

Cumulative Impact: 69% Worse Health Gap: After exhausting each collateral entirely through partial liquidations:

Path	Remaining collateral	Remaining maxDebt	Remaining debt	Health gap	Bad debt thres
A (HIGH-lltv first)	col0=1000, col1=0	770	935	**165**	11.9% price dro
B (LOW-lltv first)	col0=0, col1=1000	860	957	**97**	7.6% price drop

Path A leaves the protocol **69% worse off** in terms of remaining health deficit. The bad debt threshold for Path A requires a 12% oracle price drop, while Path B needs only 7.6%.

Impact Explanation: **Impact: Low**.

The vulnerability does not cause immediate or guaranteed fund loss. The harm is indirect and conditional:

1. A position must be deeply unhealthy (`health ratio < lif_high × lltv_high ≈ 0.891`).
2. A liquidator must target the highest-LLTV collateral (which they are financially incentivized to do — it offers the largest `maxRepaid` per call).

3. Bad debt must then materialize through a subsequent oracle price decline on the remaining low-LLTV collateral.

The severity is Low because:

- The RCF correctly recovers positions if full `maxRepaid` is used in a single call.
- Liquidators may choose wrong ordering for efficiency reasons, not malice.
- The bad debt amplification is probabilistic, not certain.

Likelihood Explanation: **Likelihood: Medium.**

This vulnerability activates naturally during market stress:

1. **Multi-collateral positions are a core feature** — up to 16 simultaneous collateral types per position.
2. **Liquidators are economically incentivized to prefer high-LLTV collateral** — the `maxRepaid` formula yields larger repayments per call for high-LLTV collateral.
3. **Partial liquidations are common** — the RCF is specifically designed to allow partial recovery.
4. **Deeply unhealthy positions are exactly when liquidations occur** — the degradation condition is met precisely when positions are significantly underwater.

The NatSpec acknowledges liquidator collateral preferences exist but does not identify the health-degradation consequence.

Recommendation:

Short-Term: Documentation: Add to the `liquidate()` NatSpec:

```
/// @dev MULTI-COLLATERAL ORDERING NOTE: In positions with multiple collaterals of
/// different LLTV values, partial liquidations of the highest-LLTV collateral can
/// DECREASE the overall health ratio when: lif_j × lltv_j > maxDebt_basket / debt.
/// Liquidation bots should target the LOWEST-LLTV collateral first to maximize
/// health ratio improvement per unit of debt repaid.
```

Medium-Term: Basket-Weighted `maxRepaid`: Replace the per-collateral LLTV in the RCF formula with a basket-weighted effective LLTV:

```
// Compute basket-weighted effective LLTV
uint256 totalCollateralValue = 0;
// (computed in oracle loop above)

uint256 effectiveLltv = maxDebt.mulDivDown(WAD, totalCollateralValue);

uint256 maxRepaid = effectiveLltv < WAD
    ? (_position.debt - maxDebt).mulDivUp(WAD * WAD, WAD * WAD - lif *
    ↪  effectiveLltv)
    : type(uint256).max;
```

This ensures the RCF caps repayment at an amount that genuinely recovers the position's basket health ratio, regardless of which specific collateral is being liquidated.

Alternative: Minimum Health Ratio Improvement Guard:

```
// Require health ratio to improve or equal after every partial liquidation
uint256 newHealthRatio = newMaxDebt.mulDivDown(WAD, _position.debt);
uint256 oldHealthRatio = maxDebt.mulDivDown(WAD, originalDebt);
require(
    repaidUnits == maxRepaid || newHealthRatio >= oldHealthRatio,
```

```
    HealthRatioMustImprove()
);
```

This prevents health-degrading partial liquidations while still allowing full-recovery `maxRepaid` calls.

Morpho: Morpho acknowledged that liquidators may have a systemic preference to liquidate certain collateral first, which is documented in the NatSpec at `Midnight.sol:37`. The health-degradation behavior described in this finding is downstream of that documented design, and Morpho considers it clear to lenders entering a given market.

Cantina Managed: Acknowledged.

3.1.7 Malicious Ratifier Can Gas-Bomb Public Bundle Fills Before Later Offers Are Reached

Severity: Low Risk.

Context: *(No context files were provided by the reviewer).*

Submitted by Kingnull, also found by deyu.

Summary: `supplyCollateralAndSellWithUnitsTarget()` tries each offer in a loop and catches reverts, but it does not recover the gas spent inside the ratifier call that triggered the revert. A malicious maker can publish public offers that point to a gas-burning ratifier. When a victim bundle includes several of those offers before a valid one, the bundle can run out of gas and revert before it reaches the valid offer.

Description: The issue is in the bundle loop around `take()`:

1. The taker supplies a `takes[]` array of public offers.
2. For each offer, the bundler computes consumable units and then calls `Midnight.take()`.
3. `take()` dispatches into the offer's ratifier before the `try/catch` in the bundler can finish.
4. A malicious ratifier can burn almost all remaining gas and then revert.
5. The bundler catches the revert, but the gas spent inside the ratifier is already gone.
6. After enough malicious offers, the transaction can run out of gas before later valid offers are even attempted.

Simple Explanation: If the attacker uses the gas-bomb offer in their own bundle, they mostly hurt themselves.

The real problem is broader: the offers are public. Any third party who tries to fill a bundle that contains those offers can lose enough gas that the whole transaction reverts before the later valid offer is reached.

Impact Explanation: This is a denial of service on the bundle fill path.

The direct effect is wasted caller gas and a reverted bundle fill. The victim does not get the intended offer execution, and the valid offer later in the array may never be reached. i will also need to include that There is no direct protocol accounting loss in the demonstrated path, so this stays below the fund-loss classes since is DOS the impact stays medium at best low impact.

Likelihood Explanation: The trigger is simple once an attacker can publish a public offer:

- The attacker points the offer at a ratifier that burns gas and reverts,
- The victim uses a bundle with several such offers in front of a valid one,
- The bundler iterates into the gas bomb before it can reach the valid offer.

No privileged role is needed. The attack depends only on public offer creation and the existing bundle loop.

Setup: Use the regression in `test/MidnightBundlesTest.sol`.

Relevant helper:

- `GasBombRatifier` in `test/MidnightBundlesTest.sol`.

Relevant test:

- `test/MidnightBundlesTest.sol::testSellUnitsTargetCanBeGasBombedByRatifier`

Recommendation: Do not let untrusted ratifier execution consume the main bundle's gas budget before later offers are evaluated.

At minimum, move the ratifier check into a capped-gas preflight or otherwise bound the per-offer gas that a malicious ratifier can consume before the bundler decides whether to continue to later offers. If the intended behavior is to skip bad offers, the skip path needs to fail fast rather than let one offer exhaust the transaction.

Morpho: Acknowledged.

Cantina Managed: Acknowledged.

3.1.8 Collateral composition can be changed post-maturity, allowing borrowers to delay liquidation

Severity: Low Risk.

Context: *(No context files were provided by the reviewer).*

Submitted by kvar, also found by Yang and 0xorange.

Description: Every market has a maturity. Once maturity has passed, borrow positions should be liquidatable. However, `Midnight.sol::withdrawCollateral()` only performs a health check and does not check whether the market has matured. This allows a borrower to delay post-maturity liquidation:

1. A borrower has a position collateralized with the token at index 0, and the market has matured.
2. A liquidator attempts to liquidate the borrower and seize collateral at index 0.
3. The borrower front-runs the liquidation by supplying collateral at index 1 and withdrawing all collateral at index 0.
4. The liquidator's transaction reverts because the borrower no longer has sufficient collateral at index 0.
5. The borrower can repeat this whenever another liquidation is submitted.

Impact Explanation: Medium impact. Post-maturity liquidations are necessary because lenders expect to redeem their credit after maturity, which requires outstanding debt to be repaid or liquidated.

By repeatedly preventing liquidation, a borrower can delay debt repayment and therefore cause a denial of service for both liquidations and lender withdrawals.

Likelihood Explanation: High likelihood, as in some markets the attack can be performed completely for free besides gas costs. The attacker only needs a way to convert between collateral tokens without fees. One example is the fee-free, bidirectional DAI/USDS converter, combined with the ability to wrap DAI into sDAI and USDS into sUSDS without conversion fees. Therefore, the attacker can rotate between four distinct collateral tokens without paying swap fees.

For example:

1. Alice backs her borrow with DAI.
2. A liquidator attempts to liquidate her position after maturity.
3. Alice front-runs the liquidation, takes a fee-free DAI flash loan, such as through Balancer, converts the DAI into USDS, supplies the USDS as collateral, withdraws her original DAI collateral, and repays the flash loan.
4. Alice's position is now backed by USDS at a different collateral index, causing the liquidator's transaction to revert.

Alice can similarly rotate between DAI, USDS, sDAI, and sUSDS to invalidate subsequent liquidations targeting her currently active collateral index.

Recommendation: Prevent borrowers with outstanding debt from withdrawing collateral after the market has matured.

Morpho: Morpho agreed with the technical description of the mechanism: `withdrawCollateral()` enforces only a health check and not maturity, so a borrower can move collateral out of the index a liquidator is targeting and into another, causing liquidations that name a specific index to revert. Morpho assessed the attack as unlikely and self-defeating because the post-maturity liquidation incentive increases monotonically with elapsed time, making delay value-destroying for the borrower. Morpho also noted that liquidators routing through a smart contract can read the borrower's current position at execution time and neutralize the front-run.

Cantina Managed: Acknowledged.

3.2 Informational

3.2.1 Lenders can wash-transfer credit after a fee cut to reset fixed future continuous fees

Severity: Informational.

Context: *(No context files were provided by the reviewer).*

Submitted by Dinesh777, also found by Kassi, magickenn, fuddleyu, hartmade, Drothon and SolidGold-Magikarp.

Summary: Midnight documents that continuous-fee changes should not impact existing lending positions. The whitepaper says lenders crystallize future fees when increasing credit, so later continuous-fee changes do not impact existing lending positions. The source NatSpec says the pending fee of existing lenders is fixed when the market's continuous fee changes.

However, an existing lender can sell the credit to a second address they control after the fee setter lowers the market continuous fee. The seller's proportional `pendingFee` is removed from the old position, while the controlled buyer receives the same credit with `pendingFee` computed from the new lower market fee. If the new fee is zero, the future continuous fee is fully reset. If the new fee is only reduced, the lender still resets the old fixed fee to the lower fee.

The proof of concept shows this remains profitable even when the market charges the maximum 360-day settlement fee: on a 1,000,000 unit position, the old fixed continuous fee is `9,863.013672576e21` units, the max settlement fee paid by the wash trade is `5,000e21` units, and the net fee avoided is `4,863.013672576e21` units. It also realizes the result after borrower repayment and lender withdrawal at maturity, proving the controlled addresses end with `4,863.013672576e21` more loan tokens. A separate test lowers the fee to one quarter of the original value, not zero, and still realizes `2,397.26027776e21` more loan tokens after max settlement fees.

The proof of concept also includes a path through the in-scope `SetterRatifier` contract instead of relying only on the test helper ratifier, proving the behavior survives the production offer-ratification flow.

Root Cause: When a lender sells credit through `take`, the seller's fixed pending fee is removed proportionally:

- `src/Midnight.sol:387`
- `src/Midnight.sol:412`

The buyer's pending fee is not inherited from the seller. It is recomputed from the current market continuous fee:

- `src/Midnight.sol:385`
- `src/Midnight.sol:409`

This means a transfer of already-existing credit is treated like new credit for continuous-fee purposes.

The relevant documented expectation is:

- `src/Midnight.sol:51`
- `src/Midnight.sol:53`
- `src/Midnight.sol:54`
- `tmp/midnight-whitepaper.txt:503`
- `tmp/midnight-whitepaper.txt:505`

Attack Path:

1. A lender enters a market while `continuousFee` is high.
2. The fee setter later lowers the market continuous fee.
3. The lender creates or controls a second address.
4. The second address posts a buy offer for the lender's full credit.
5. The original lender takes that offer and sells the credit to the controlled address.
6. The original lender's fixed `pendingFee` is cleared.
7. The controlled buyer receives the same credit with `pendingFee` based on the new lower fee.
8. At maturity, no old future continuous fee materializes.

Impact: Protocol continuous-fee revenue can be avoided by users who already had fixed pending fees before a fee cut. At maturity, the old fixed pending fee no longer materializes as `continuousFeeCredit`, so the fee claimer receives less than the documented fee model implies.

The strengthened proof of concept proves the impact as final loan-token balances after borrower repayment and withdrawal at maturity, not only as an intermediate accounting delta. It also proves the issue is not limited to disabling the fee completely; a partial fee reduction remains profitable under the maximum long-dated settlement fee.

Scope and Validity: This is a contract-level accounting flaw in the in-scope `src/Midnight.sol` credit-transfer path.

- The affected source file is in scope.
- The reset is reachable by an ordinary lender after a normal market fee decrease.
- The issue does not require compromising the fee setter, borrower, ratifier, fee claimer, or any privileged account.
- The behavior contradicts the whitepaper/NatSpec statement that later continuous-fee changes do not impact existing lending positions.
- The proof of concept uses the real `take`, `updatePosition`, `repay`, and `withdraw` flows.
- The proof of concept includes an in-scope `SetterRatifier` flow, so it does not depend only on a test-only ratifier helper.
- The proof of concept quantifies realized fee avoidance after max long-dated settlement fees.
- The proof of concept includes a partial-fee-cut case, so the exploit does not depend on setting the market continuous fee to zero.

Proof of Concept: Proof of concept file:

```
/home/dinesh/cantina/morpho-midnight-audit/midnight/test/ContinuousFeeWashTransferP
↪  oC.t.sol
```

Run from:

```
/home/dinesh/cantina/morpho-midnight-audit/midnight
```

Command:

```
forge test --match-path test/ContinuousFeeWashTransferPoC.t.sol --match-test
↪  'testLenderCanResetFixedFutureContinuousFeeByWashTransferringCreditAfterFeeCut|
↪  testSetterRatifierPathAlsoResetsFixedFutureContinuousFee|testWashTransferRemain
↪  sProfitableEvenWithMaxLongDatedSettlementFee|testWashTransferProducesHigherReal
↪  izedTokenBalanceAfterRepayAndWithdraw|testPartialFeeCutStillRealizesProfitAfter
↪  RepayAndWithdraw' -vvv
```

Observed output:

```
[⠊] Compiling...
No files changed, compilation skipped

Ran 5 tests for test/ContinuousFeeWashTransferPoC.t.sol:ContinuousFeeWashTransferPoC
[PASS]
↪  testLenderCanResetFixedFutureContinuousFeeByWashTransferringCreditAfterFeeCut()
↪  (gas: 1062126)
Logs:
  fee avoided by wash transfer: 9863013672576000000000

[PASS] testPartialFeeCutStillRealizesProfitAfterRepayAndWithdraw() (gas: 1679192)
Logs:
  old fixed pending fee: 9863013672576000000000
  new reduced pending fee: 2465753394816000000000
  max settlement fee paid: 500000000000000000000
  partial-cut realized token gain: 2397260277760000000000

[PASS] testSetterRatifierPathAlsoResetsFixedFutureContinuousFee() (gas: 1563986)
[PASS] testWashTransferProducesHigherRealizedTokenBalanceAfterRepayAndWithdraw()
↪  (gas: 1675988)
Logs:
  baseline controlled token balance: 1990136986327424000000000
  wash controlled token balance: 1995000000000000000000000
  realized token gain after repay and withdraw: 4863013672576000000000

[PASS] testWashTransferRemainsProfitableEvenWithMaxLongDatedSettlementFee() (gas:
↪  1066246)
Logs:
  fixed continuous fee avoided: 9863013672576000000000
  max settlement fee paid: 500000000000000000000
  net fee avoided: 4863013672576000000000

Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 15.19ms (29.66ms CPU
↪  time)

Ran 1 test suite in 29.79ms (15.19ms CPU time): 5 tests passed, 0 failed, 0 skipped
↪  (5 total tests)
```

Recommendation: If existing-credit fee invariance is intended to survive credit transfers, carry the seller's proportional `pendingFee` to the buyer when an existing credit position is transferred.

One approach:

- Split `take` accounting between credit transferred from the seller and newly originated buyer credit.
- For the transferred-credit portion, add the seller's proportional `pendingFeeDecrease` to the buyer instead of clearing it.
- For genuinely new credit, keep using the current market continuous fee.

If the current behavior is intentional, document explicitly that existing lenders can reset future continuous fees by exiting and re-entering through a controlled buyer after fee decreases.

Morpho: Acknowledged.

Cantina Managed: Acknowledged.

3.2.2 Canceled root check occurs after Merkle proof verification

Severity: Informational.

Context: *(No context files were provided by the reviewer).*

Submitted by Athenea, also found by santin0x.

Summary: `EcrecoverRatifier.isRatified()` verifies the Merkle proof before checking whether the root was canceled, causing unnecessary gas usage for calls using already-canceled roots.

Description: In `EcrecoverRatifier.isRatified()`, the function first verifies the Merkle proof:

```
require(
    HashLib.isLeaf(root, HashLib.hashOffer(offer), leafIndex, proof),
    InvalidProof()
);
```

Only after that does it check whether the root was canceled:

```
require(!isRootCanceled[offer.maker][root], RootCanceled());
```

For a valid offer proof under a canceled root, the function performs the full proof traversal before reverting. The cancellation check is a simple storage read and could be performed earlier after decoding `ratifierData`.

This does not allow theft or unauthorized execution. It only breaks the expected gas-efficiency property that obviously invalid canceled roots should be rejected as early as possible.

Impact Explanation Impact is Low. No funds are lost, no valid offer is blocked beyond the intended cancellation behavior, and an attacker cannot force other users to pay gas. The only impact is extra gas spent by callers who submit `take()` transactions using a canceled root.

Impact Explanation: Elaborate on why you've chosen a particular impact assessment.

Likelihood Explanation: Likelihood is Low. This only happens when a caller submits a valid Merkle proof for a root that has already been canceled. Normal integrations can avoid this by checking `isRootCanceled(maker, root)` off-chain or on-chain before calling `take()`.

Recommendation: Check root cancellation before verifying the Merkle proof:

```
function isRatified(
    Offer memory offer,
    bytes memory ratifierData
) external view returns (bytes32) {
    require(msg.sender == MIDNIGHT, NotMidnight());
```

```
        (
            Signature memory sig,
            bytes32 root,
            uint256 leafIndex,
            bytes32[] memory proof
        ) = abi.decode(ratifierData, (Signature, bytes32, uint256, bytes32[]));

        require(!isRootCanceled[offer.maker][root], RootCanceled());

        require(
            HashLib.isLeaf(root, HashLib.hashOffer(offer), leafIndex, proof),
            InvalidProof()
        );

        bytes32 structHash = keccak256(
            abi.encode(HashLib.offerTreeTypeHash(proof.length), root)
        );

        bytes32 domainSeparator = keccak256(
            abi.encode(EIP712_DOMAIN_TYPEHASH, block.chainid, address(this))
        );

        bytes32 digest = keccak256(
            bytes.concat("\x19\x01", domainSeparator, structHash)
        );

        address signer = ecrecover(digest, sig.v, sig.r, sig.s);
        require(signer != address(0), InvalidSignature());
        require(
            signer == offer.maker ||
                IMidnight(MIDNIGHT).isAuthorized(offer.maker, signer),
            Unauthorized()
        );

        return CALLBACK_SUCCESS;
    }
```

Morpho: Morpho addressed the check ordering in `EcrecoverRatifier.isRatified()` by moving the `isRootCanceled` storage read before the Merkle proof traversal. The fix was merged in pull request #1011.

Cantina Managed: Fix verified.

3.2.3 Deviation in oracle price could lead to arbitrage in high LLTV markets

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Submitted by 0xG0P1, also found by 0xAristos and luciomagnn.

Description:

Oracle Front-Running Can Create Bad Debt Through Temporary Price Deviations: In Morpho Midnight, the maximum amount a user can borrow is determined using the conversion rate between the `loanToken` and `collateralToken` returned by the market oracle.

However, all oracle designs are susceptible to temporary pricing discrepancies because oracle prices inherently lag behind real-time market prices.

Sources of Oracle Lag:

- **Chainlink oracles** only update when the asset price moves beyond a predefined deviation threshold. As a result, the reported price can be slightly higher or lower than the actual market price.

- **Uniswap V3 TWAP oracles** return an average price over a historical window, causing the reported price to lag behind current market conditions.

An attacker can exploit these temporary discrepancies by front-running an oracle update and opening a position using stale prices. Once the oracle updates, the position may immediately become undercollateralized and generate bad debt.

For Morpho Midnight, this attack becomes profitable whenever:

$$\text{Price Deviation} > \frac{1}{LIF} - LLTV$$

The likelihood of this condition being satisfied increases significantly when `ChainlinkOracle.sol` is configured using multiple Chainlink price feeds.

When several feeds are chained together to derive the final conversion rate, each feed contributes its own deviation threshold. The cumulative effect can result in a substantially larger overall pricing error.

Example: Assume a market where:

- `loanToken` = ETH.
- `collateralToken` = WBTC and SOL.

Current market prices:

Asset	Price
BTC	60,000 USD
ETH	1,600 USD
SOL	60 USD

The oracle uses the following Chainlink feeds:

Feed	Deviation Threshold
WBTC / BTC	0.5%
BTC / ETH	2.0%
SOL / USD	2.0%
ETH / USD	0.5%

Assume every feed is currently sitting exactly at its maximum allowed deviation threshold.

WBTC → ETH Conversion: Actual value:

```
1 WBTC = 60000 / 1600
       = 37.5 ETH
```

Applying oracle deviations:

```
1 WBTC = 37.5
       * (1 - 0.005)
       * (1 - 0.02)
       / (1 + 0.005)

1 WBTC = 36.56625 ETH
```

SOL → ETH Conversion: Actual value:

```
1 SOL = 60 / 1600
      = 0.0375 ETH
```

Applying oracle deviations:

```
1 SOL = 0.0375
      * (1 - 0.02)
      / (1 + 0.005)

1 SOL = 0.03656625 ETH
```

Total Collateral Value: According to the stale oracle:

```
36.56625 + 0.03656625
= 36.60281625 ETH
```

Actual market value:

```
(60000 + 60) / 1600
= 37.5375 ETH
```

Price deviation:

```
(37.5375 - 36.60281625) / 37.5375
= 0.0249
= 2.49%
≈ 2.5%
```

Exploiting the Deviation: Assume:

- LLTV = 98%.
- Collateral = 1 WBTC + 1 SOL.

At the time of borrowing, the market still values the collateral at:

```
1 WBTC + 1 SOL
= 37.5375 ETH
```

The attacker deposits:

```
1 WBTC + 1 SOL
```

and borrows the maximum allowed amount:

```
37.5375 * 98%
= 36.78675 ETH
```

Oracle Update: Immediately afterward, the oracle updates and the collateral value becomes:

```
1 WBTC + 1 SOL
= 36.60281625 ETH
```

The borrower's debt is now larger than the value of the collateral:

```
36.78675 ETH > 36.60281625 ETH
```

The position instantly becomes unhealthy and contains bad debt.

Self-Liquidation: At 98% LLTV:

```
LIF = 100.5%
    = 1.005
```

To seize the entire collateral, the liquidator only needs to repay:

```
36.60281625 / 1.005
= 36.42071 ETH
```

Thus:

Borrowed amount:

```
36.78675 ETH
```

Repayment amount:

```
36.42071 ETH
```

Profit:

```
36.78675 - 36.42071
= 0.36604 ETH
```

Note that Morpho midnight only allows to liquidate partial amount of collateral , so in self liquidation the borrower initially liquidates partially and then after the maturity he can liquidate the remaining collateral to get the profit.

Impact: The attacker earns:

```
0.36604 ETH
```

without taking meaningful market risk.

The unpaid portion of the debt remains in the market as bad debt and is ultimately socialized across lenders.

As a result, any profit extracted through this oracle-lag arbitrage directly translates into losses for lenders.

Recommendation:

Non trivial.

Morpho: Acknowledged.

Cantina Managed: Acknowledged.